Lesson1_3_7

September 4, 2026

1 Introduction

This web site is running a "Jupyter notebook," which allows you to execute interactive Octave code directly in your browser. Each (part of a code) line preceeded by a percent symbol "%" is a code comment.

Reminder: To execute any Octave commands, first enter the commands in a code cell, and then depress the "shift" and "enter" keys on your keyboard at the same time, making sure that your cursor is placed inside of the code cell. I refer to this as "$<$ shift $>$ $<$ enter $>$" in my instructions below.

The output from this notebook will be used to provide answers for this lesson's practice quiz. As you proceed through the specialization, you will use more and more Jupyter notebooks to write Octave code to implement Kalman-filter algorithms.

[5]:
```
% Place your cursor in this input code cell. Then, type < shift >< enter > to␣
↪execute

% We require the Octave control-systems "package" to implement "expm".
pkg load control
```

1.0.1 Converting continuous-time noise descriptions to discrete-time

The next notebook cells allow you to convert from continuous-time noise descriptions to discrete-time easily.

[6]:
```
% Define the constants of the continuous-time noise model
A = -0.5; % continuous-time system "A" matrix
B = 0.2;  % continuous-time system "B" matrix
C = 1;    % continuous-time system "C" matrix
D = 0;    % continuous-time system "D" matrix

Bw = 0.5; % Continuous-time system "B" matrix for process-noise input
Sw = 1;   % continuous-time process-noise spectral density
Sv = 0.1; % continuous-time sensor-noise spectral density

dT = 0.01; % sampling period
```

[7]:

```
% Compute the conversions
Z = [-A, Bw*Sw*Bw'; 0*A, A'];
CC = expm(Z*dT);
[nx,nu] = size(Bw);
c11 = CC(1:nx,1:nx);
c12 = CC(1:nx,nx+1:end);
c21 = CC(nx+1:end,1:nx);
c22 = CC(nx+1:end,nx+1:end);

Ad = c22'
SigmaW = Ad*c12
SigmaV = Sv/dT
```

```
Ad =  0.99501
SigmaW =  0.0024875
SigmaV =  10
Ad =  0.99501
SigmaW =  0.0024875
SigmaV =  10
```